

RTOS-Based Auxiliary Safety Monitoring System for Nuclear Power Plant Applications Using ESP32 and FreeRTOS

Real Time Operating Systems 25ES611

PROJECT REPORT

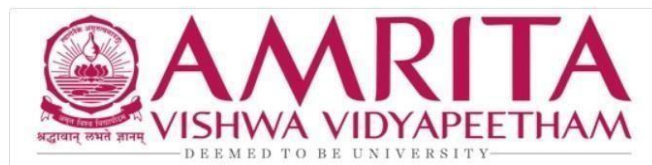
Submitted by

BL.EN.P2EBS25017 THARUNKUMAR R

Guided by

Dr. Surekha P

ELECTRICAL AND ELECTRONICS ENGINEERING



**AMRITA SCHOOL OF ENGINEERING,
BENGALURU AMRITA VISHWA VIDYAPEETHAM**

BENGALURU

560035

April 2026

Table of Contents

CHAPTER 1: INTRODUCTION	4
1.1 Problem Statement	4
1.2 Objectives	5
1.3 Novelty and Socio-economic Relevance	5
CHAPTER 2: LITERATURE SURVEY	6
2.1 Real-Time Monitoring Systems using RTOS	6
2.2 Embedded Systems and FreeRTOS Applications.....	6
2.3 Scheduling, Timing and Priority Analysis	6
2.4 Research Gap	6
Existing studies mainly focus on functional monitoring or cloud integration. Limited work combines:	6
CHAPTER 3: METHODOLOGY	7
3.1 System Block Diagram	8
3.2 Working Principle	9
3.3 Dashboard Integration.....	9
Streamlit Dashboard.....	9
Serial Studio.....	9
3.4 Queue-Based Communication	10
3.6 System Operation.....	11
3.7 Threshold Conditions.....	12
3.8 Task Scheduling Strategy	12
4.1 Scenario 1: Normal Monitoring Condition	13
Discussion	13
4.2 Scenario 2: Radiation Alert Detection	13
Discussion	14

4.3 Scenario 3: MQ2 Hazard Detection	14
Discussion	14
4.4 Scenario 4: Temperature Alert Detection	14
Discussion	14
4.5 Scenario 5: Dashboard Integration.....	15
Discussion	15
Discussion	16
4.7 Quantitative Performance Metrics	16
4.8 Discussion	17
CONCLUSION.....	19
REFERENCES	21

CHAPTER 1: INTRODUCTION

Real-time monitoring systems play a vital role in safety-critical environments such as nuclear power plants where hazardous situations require immediate response. Conventional monitoring approaches often rely on sequential execution and polling mechanisms which may introduce delays during emergency conditions. Such systems may not ensure deterministic behavior under multiple simultaneous events.

This project presents an RTOS-Based Auxiliary Safety Monitoring System for Nuclear Power Plant Applications using ESP32 and FreeRTOS. The proposed system continuously monitors radiation level, hazardous gas concentration, and environmental temperature using concurrent task execution. A potentiometer is used to simulate radiation variation while an MQ-2 sensor detects hazardous gas levels. Temperature monitoring is performed using a DHT11 sensor.

FreeRTOS enables priority-based task scheduling, queue communication, and deterministic execution. A centralized monitoring dashboard is developed using Streamlit while Serial Studio is used to visualize timing and scheduling behavior.

The system prioritizes events according to severity:

Radiation > MQ2 Hazard > Temperature

This ensures that critical conditions receive immediate attention.

1.1 Problem Statement

Safety monitoring systems in industrial environments require simultaneous monitoring of multiple parameters with minimal delay. Conventional embedded systems often focus mainly on functional operation while providing limited timing guarantees. Delayed detection of hazardous conditions can affect system reliability, responsiveness, and operational safety.

Traditional monitoring approaches often use sequential processing methods, which may introduce delays when handling multiple events simultaneously. Safety-critical applications require deterministic execution and priority-based scheduling to ensure immediate processing of critical events.

The problem addressed in this work is the development of a low-cost embedded system capable of real-time monitoring with deterministic task scheduling, simultaneous parameter observation, and priority-based event handling for improved safety and responsiveness.

1.2 Objectives

- To monitor radiation levels using a potentiometer-based simulation and continuously observe variations in radiation intensity through analog sensing techniques.
- To detect hazardous gas conditions using the MQ-2 sensor and generate alerts whenever gas concentration exceeds predefined threshold values.
- To monitor environmental temperature using the DHT11 sensor and identify abnormal temperature variations that may affect system safety.
- To implement real-time task scheduling using FreeRTOS for simultaneous execution of multiple sensing and processing tasks.
- To develop a priority-based alert generation mechanism where critical events are processed according to their severity level.
- To ensure deterministic task execution through preemptive scheduling and efficient resource management.
- To implement queue-based inter-task communication for synchronization and reliable data transfer between sensing and processing modules.
- To create centralized dashboards using Streamlit for real-time visualization of sensor activity and system status.
- To integrate Serial Studio for monitoring RTOS timing behavior and task execution characteristics.
- To analyze system timing performance, scheduling behavior, and multitasking capability under different operating conditions.

1.3 Novelty and Socio-economic Relevance

The proposed system combines multiple sensing modalities with RTOS-based scheduling in a single integrated architecture for simultaneous monitoring of radiation levels, hazardous gas concentration, and environmental temperature. Priority-driven event handling enables immediate response to critical hazards through deterministic scheduling and concurrent task execution using FreeRTOS. The use of ESP32 and low-cost sensors significantly reduces implementation cost while maintaining reliability and scalability. Integration of real-time dashboards using Streamlit and Serial Studio enhances monitoring and timing analysis capabilities. From a socio-economic perspective, the system provides a cost-effective and scalable solution suitable for industrial safety applications, research environments, and educational platforms requiring real-time monitoring and rapid hazard response.

CHAPTER 2: LITERATURE SURVEY

2.1 Real-Time Monitoring Systems using RTOS

Alves et al. [1] proposed a FreeRTOS-based real-time air quality monitoring architecture and demonstrated improved transmission performance and reduced latency using RTOS scheduling techniques. Their work highlighted the benefits of deterministic execution in monitoring systems.

Janeera et al. [2] developed an embedded IoT framework for real-time environmental monitoring and alert generation. The work demonstrated the integration of embedded sensing and communication technologies for monitoring applications.

Wang et al. [3] implemented an IoT-based real-time health monitoring architecture for multi-scenario data acquisition and emphasized continuous monitoring and data management capabilities.

Zhu et al. [4] proposed an RTOS-based wireless monitoring system that employed distributed sensing nodes and remote data visualization techniques. Their work highlighted scalable monitoring architectures.

2.2 Embedded Systems and FreeRTOS Applications

Lin and Wang [5] analyzed FreeRTOS scheduling behavior using formal verification approaches and demonstrated the impact of scheduling configurations on system correctness.

Calligaris et al. [6] presented OpenIPMC, a monitoring and control framework based on FreeRTOS supporting ESP32 and other platforms for embedded system management applications.

Reddy et al. [7] presented a comprehensive review of real-time operating systems and discussed task management and deterministic scheduling approaches in embedded applications.

Septian et al. [8] proposed a FreeRTOS-based air-quality monitoring platform integrated with secure IoT communication methods. Their study demonstrated RTOS capability for reliable monitoring applications.

2.3 Scheduling, Timing and Priority Analysis

FreeRTOS scheduling policies play a significant role in embedded systems requiring deadline-aware execution. Lin and Wang [9] demonstrated that scheduler configurations significantly affect timing correctness and response behavior.

Recent RTOS-based monitoring architectures have shown improvements in transmission intervals and throughput due to deterministic task scheduling mechanisms.

2.4 Research Gap

Existing studies mainly focus on functional monitoring or cloud integration. Limited work combines:

- FreeRTOS task prioritization
- Simultaneous hazard monitoring
- Real-time dashboard visualization
- Timing analysis using lightweight tools
- Low-cost ESP32 implementation

CHAPTER 3: METHODOLOGY

The proposed RTOS-Based Auxiliary Safety Monitoring System for Nuclear Power Plant Applications is designed to monitor multiple environmental parameters simultaneously using a multitasking architecture implemented on ESP32 with FreeRTOS. The system continuously acquires radiation level, hazardous gas concentration, and temperature values and processes them using priority-based task scheduling.

The implementation uses a **Potentiometer** to simulate radiation variation, an **MQ-2 sensor** for hazardous gas detection, and a **DHT11 sensor** for temperature monitoring. These sensors are interfaced with the ESP32 microcontroller which acts as the central processing unit. FreeRTOS provides concurrent task execution, queue-based communication, and deterministic scheduling.

The methodology follows the sequence below:

1. Sensor values are continuously acquired from the potentiometer, MQ-2, and DHT11 sensors.
2. Separate FreeRTOS tasks are created for each sensing operation.
3. Priorities are assigned based on hazard severity.
4. Sensor information is transmitted through queue structures.
5. Alert conditions are generated automatically when thresholds are exceeded.
6. Dashboard and timing visualization tools display system behavior.

The task priority assignment is:

Task	Priority
Radiation Task	4
MQ2 Hazard Task	3
Temperature Task	2
Alert and Dashboard Task	1

Radiation monitoring is assigned the highest priority because radiation hazards require immediate system response. MQ2 hazard monitoring executes next followed by temperature monitoring. Lower-priority tasks execute only when higher-priority conditions are absent.

3.1 System Block Diagram

RTOS BASED AUXILIARY SAFETY MONITORING SYSTEM (ESP32 – FreeRTOS Based System)

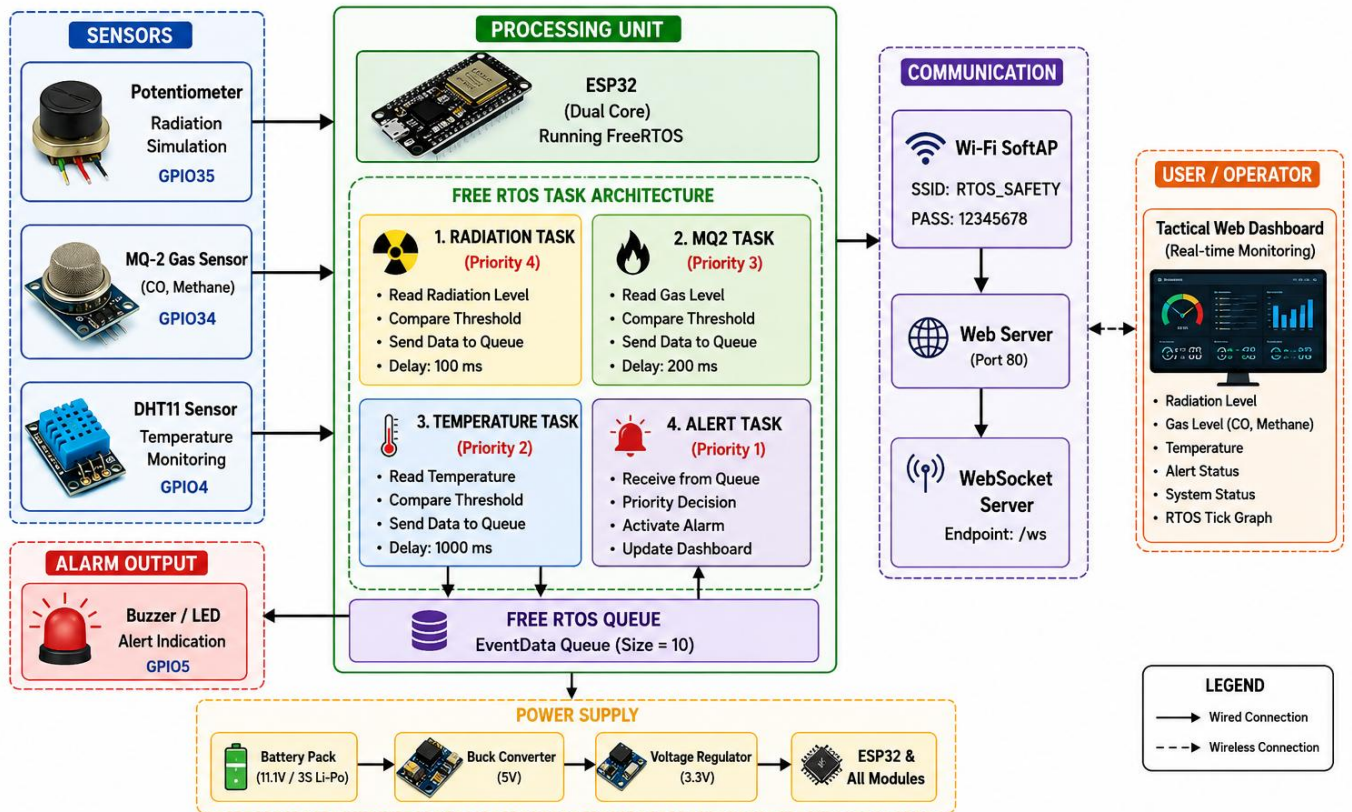


Figure 1: System Block Diagram of ESP32 RTOS based Auxiliary Safety Monitoring System

The block diagram illustrates the architecture of the proposed **RTOS-Based Auxiliary Safety Monitoring System using ESP32 and FreeRTOS**. The system consists of three sensing modules: a potentiometer for radiation simulation, an MQ-2 sensor for hazardous gas detection, and a DHT11 sensor for temperature monitoring. These sensors are interfaced with the ESP32 microcontroller, which executes multiple FreeRTOS tasks with assigned priorities. The Radiation Task is given the highest priority, followed by MQ2 and Temperature tasks, while the Alert Task processes queue data and generates warning actions. Sensor information is exchanged using a FreeRTOS EventData Queue for efficient inter-task communication. The system activates a buzzer during emergency conditions and simultaneously updates a real-time dashboard through Wi-Fi communication. A web-based monitoring dashboard displays sensor readings, alert status, and RTOS timing information, enabling centralized visualization and effective monitoring of system behavior.

3.2 Working Principle

Initially, the potentiometer generates an analog signal representing radiation level variation. The ESP32 reads the analog signal through GPIO35 and converts it using the internal ADC. Simultaneously, the MQ-2 sensor continuously measures gas concentration through GPIO34, while the DHT11 periodically measures environmental temperature through GPIO4.

Each sensor operates through an independent FreeRTOS task. The sensing tasks transmit acquired data to queue structures for inter-task communication.

When threshold values are exceeded:

- Radiation value > threshold → Radiation alert generated
- MQ-2 value > threshold → Hazard alert generated
- Temperature value > threshold → Temperature alert generated

The alert task compares incoming values and executes according to task priority.

Priority hierarchy:

Radiation > MQ2 Hazard > Temperature

The buzzer is activated whenever a critical condition occurs. Sensor values and system status are simultaneously transmitted to Streamlit and Serial Studio dashboards.

3.3 Dashboard Integration

Two visualization tools are integrated:

Streamlit Dashboard

The Streamlit dashboard functions as a centralized monitoring interface and displays:

- Radiation value
- MQ2 sensor value
- Temperature
- Alert status
- Simulated warning LED
- Live graphical trends

Serial Studio

Serial Studio is used for:

- RTOS Tick visualization
- Timing analysis
- Task activity monitoring
- Scheduling behavior observation

Continuous Tick variation confirms concurrent task execution and proper RTOS scheduling behavior.

3.4 Queue-Based Communication

Inter-task communication is implemented using FreeRTOS queues. Sensor tasks act as producers while the alert task acts as the consumer.

Producer:

```
xQueueSend(eventQueue, &data, 0);
```

Consumer:

```
xQueueReceive(eventQueue, &data, portMAX_DELAY);
```

Queue communication decouples sensing operations from processing tasks and improves system responsiveness.

3.5 Software Flowchart

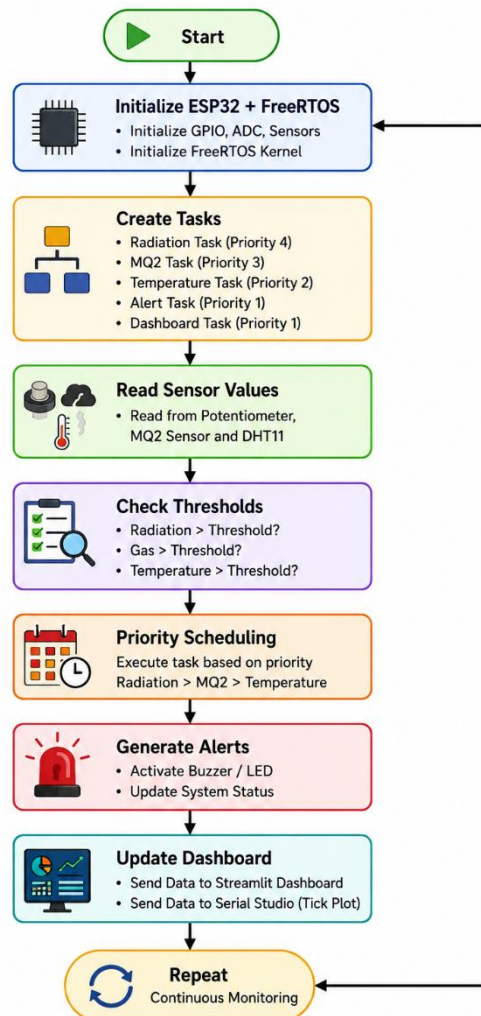


Figure 2: Flow Chart of Software System

3.6 System Operation

The proposed RTOS-Based Auxiliary Safety Monitoring System operates using concurrent task execution through FreeRTOS. Initially, the ESP32 initializes all required hardware peripherals including the MQ-2 sensor, DHT11 sensor, potentiometer, buzzer, and communication interfaces. The FreeRTOS kernel initializes task scheduling and queue management mechanisms.

Multiple tasks are created for radiation monitoring, MQ2 hazard monitoring, temperature monitoring, alert handling, and dashboard communication. Each task executes independently according to assigned priority levels.

The potentiometer simulates radiation intensity variation through analog input signals. Simultaneously, the MQ-2 sensor continuously monitors hazardous gas concentration while the DHT11 acquires environmental temperature values.

Sensor data is transmitted using queue-based communication structures. The Alert Task continuously receives and evaluates sensor values against predefined thresholds.

Task priority assignment is:

Radiation Task	→ Priority 4
MQ2 Task	→ Priority 3
Temperature Task	→ Priority 2
Alert Task	→ Priority 1
Dashboard Task	→ Priority 1

Whenever threshold conditions are exceeded, FreeRTOS immediately schedules higher-priority tasks for execution.

Radiation conditions receive maximum priority due to their safety significance, followed by MQ2 hazard conditions and temperature monitoring.

During alert situations:

- Buzzer activates automatically
- Dashboard status changes
- Simulated warning LED blinks
- Serial Studio updates timing traces
- Streamlit updates system status

The monitoring cycle repeats continuously for uninterrupted operation.

3.7 Threshold Conditions

The system continuously compares acquired sensor values against predefined threshold values. Whenever measured values exceed these limits, corresponding alert events are generated automatically.

Parameter	Threshold	Action
Radiation	>70	Radiation Alert
MQ2 Sensor	>1500	Hazard Alert
Temperature	>35°C	Temperature Alert

Threshold values were selected experimentally to ensure reliable demonstration and rapid system response.

3.8 Task Scheduling Strategy

The proposed architecture uses preemptive FreeRTOS scheduling where higher-priority tasks interrupt lower-priority execution whenever critical events occur.

Task execution order follows:

`Radiation > MQ2 > Temperature`

This scheduling mechanism ensures deterministic execution and minimizes response time during hazardous conditions.

Radiation monitoring receives the highest priority because radiation-related events require immediate processing. MQ2 hazard detection executes with secondary priority while temperature monitoring receives lower priority.

This scheduling approach improves responsiveness and guarantees timely execution of safety-related tasks.

CHAPTER 4: RESULT AND DISCUSSION

The proposed RTOS-Based Auxiliary Safety Monitoring System was implemented using ESP32 and FreeRTOS to validate deterministic task execution, simultaneous monitoring capability, and priority-based hazard processing. The system continuously monitored radiation level through a potentiometer, hazardous gas concentration using an MQ-2 sensor, and environmental temperature using a DHT11 sensor.

FreeRTOS enabled concurrent execution of sensing, processing, and dashboard tasks. The implemented system was evaluated under different operating conditions to observe task behavior, response time, and alert generation.

4.1 Scenario 1: Normal Monitoring Condition

Under normal operating conditions, sensor values remained below predefined threshold values. The Radiation Task, MQ2 Task, and Temperature Task executed periodically while maintaining deterministic scheduling behavior.

The dashboard displayed:

```
Status: NORMAL  
Buzzer: OFF  
LED: SAFE
```

The Streamlit dashboard displayed live values and Serial Studio continuously updated the RTOS Tick graph.

Discussion

During normal operation, all tasks executed according to assigned periods without generating alert conditions. The scheduler allocated CPU resources according to task priority while maintaining simultaneous monitoring.

4.2 Scenario 2: Radiation Alert Detection

Radiation levels were simulated using a potentiometer. The analog value was gradually increased to exceed the threshold value.

Condition:

```
Radiation > 70
```

When the radiation threshold was exceeded:

- Radiation Task immediately received CPU allocation
- Higher-priority scheduling activated
- Buzzer generated warning signal
- Dashboard status changed
- Simulated LED started blinking

Dashboard output:

```
Status: RADIATION ALERT  
Priority: Highest  
Buzzer: ON
```

Discussion

Since Radiation Task had the highest priority, lower-priority tasks temporarily yielded processor resources. This demonstrated preemptive scheduling and deterministic behavior.

4.3 Scenario 3: MQ2 Hazard Detection

The MQ-2 sensor was exposed to gas or smoke conditions to generate higher analog readings.

Condition:

MQ2 >1500

When hazardous gas concentration exceeded the threshold:

- MQ2 Task received processor allocation
- Alert condition generated
- Buzzer activated
- Dashboard updated system status

Dashboard output:

Status: GAS ALERT

Discussion

MQ2 hazard monitoring demonstrated the ability of FreeRTOS to process multiple environmental conditions simultaneously while preserving task priorities.

4.4 Scenario 4: Temperature Alert Detection

Environmental temperature was increased beyond predefined limits.

Condition:

Temperature >35°C

System response:

- Temperature Task generated alert event
- Dashboard displayed warning condition
- Buzzer activated

Dashboard output:

Status: TEMP ALERT

Discussion

Temperature monitoring received lower scheduling priority compared with radiation and hazard detection. Alert generation occurred only when higher-priority events were absent.

4.5 Scenario 5: Dashboard Integration

A Streamlit-based dashboard was developed for centralized monitoring and visualization. The dashboard displayed:

- Radiation level
- MQ2 gas concentration
- Temperature values
- System status
- Blinking alert LED
- Real-time graphs

Serial Studio was used to monitor RTOS Tick behavior and task timing characteristics.

Discussion

Dashboard integration provided real-time visibility into system activity and simplified monitoring.

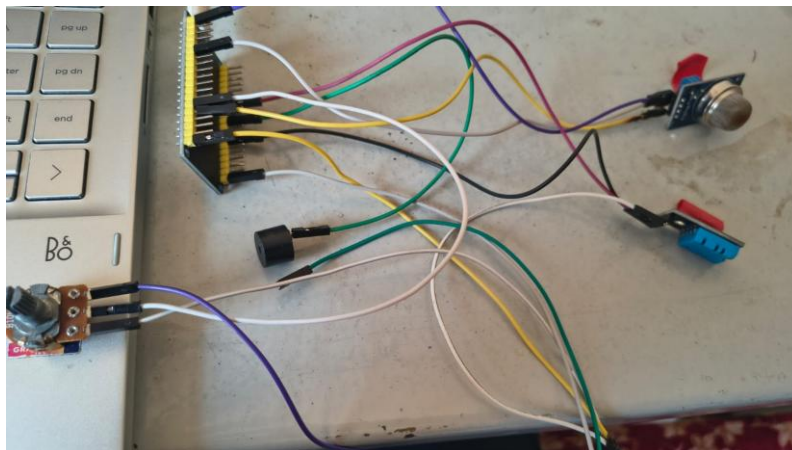


Figure 3: Hardware of ESP32 RTOS based Auxiliary Safety Monitoring System

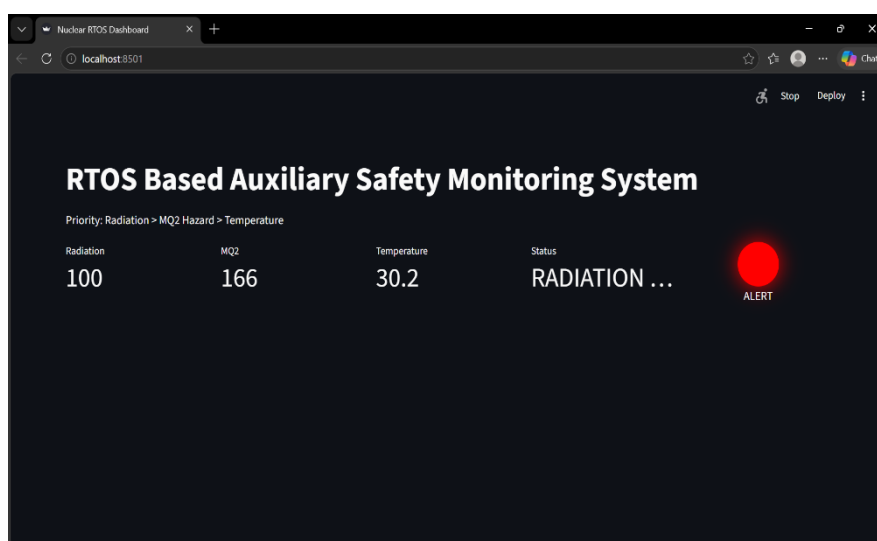


Figure 4: Dashboard of ESP32 RTOS based Auxiliary Safety Monitoring System

Serial Studio Quick Plot mode was employed for timing visualization and RTOS scheduling observation.

The Tick signal continuously increased throughout operation.

Observed signals:

Signal	Description
Radiation	Potentiometer activity
MQ2	Gas sensor variation
Temperature	Environmental condition
Tick	RTOS scheduling behavior

Continuous Tick variation confirmed:

- Simultaneous task execution
- Correct FreeRTOS scheduling
- Continuous operation
- Deterministic behavior

Discussion

Tick activity verified that multiple tasks operated concurrently and scheduling behavior remained stable.

4.7 Quantitative Performance Metrics

Task	Priority	Execution Period	Response Time
Radiation Task	4	100 ms	<100 ms
MQ2 Task	3	200 ms	<200 ms
Temperature Task	2	1000 ms	<1000 ms
Alert Task	1	Event Based	Immediate
Dashboard Task	1	1000 ms	~1 s

4.8 Discussion

The experimental results demonstrate successful implementation of deterministic scheduling and simultaneous environmental monitoring using ESP32 and FreeRTOS. The scheduler effectively handled priority-based task execution while maintaining responsiveness under multiple alert conditions.

Radiation events received immediate processor allocation due to highest priority assignment. MQ2 hazard conditions and temperature alerts were processed according to predefined scheduling order.

Integration of Streamlit and Serial Studio enabled effective monitoring and timing analysis, validating the capability of the proposed architecture for safety-critical monitoring applications.

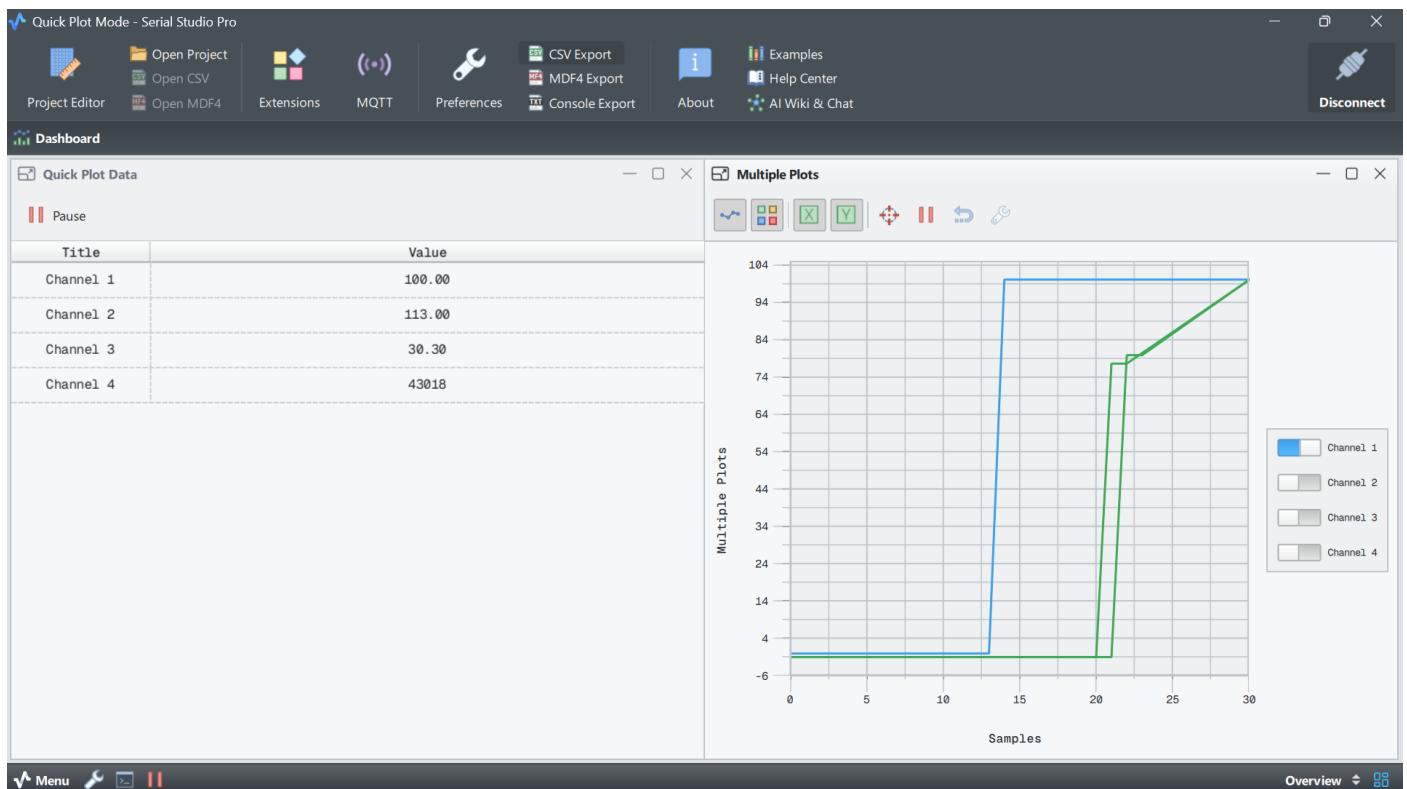


Figure 5: Timing diagram of Radiation Task

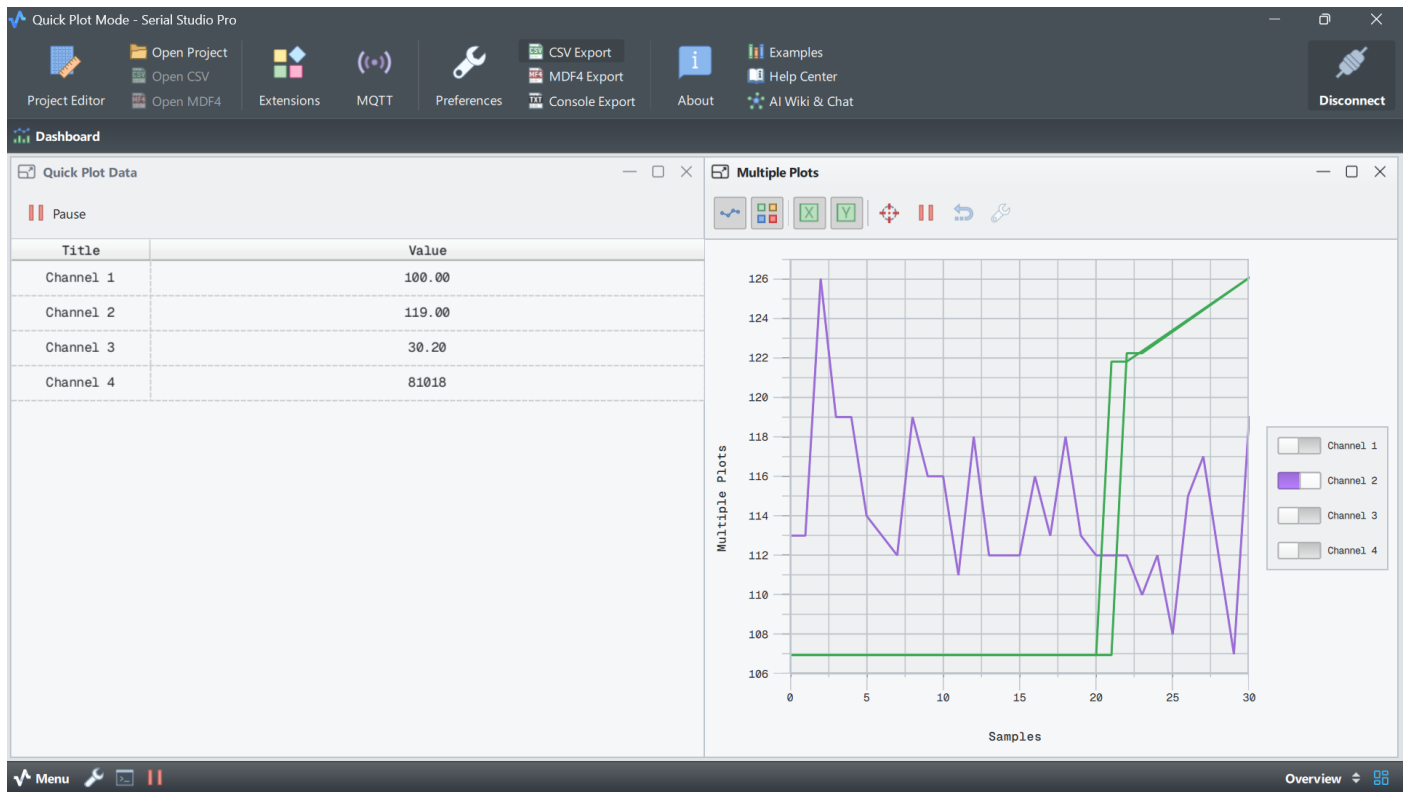


Figure 6: Timing diagram of MQ2 task

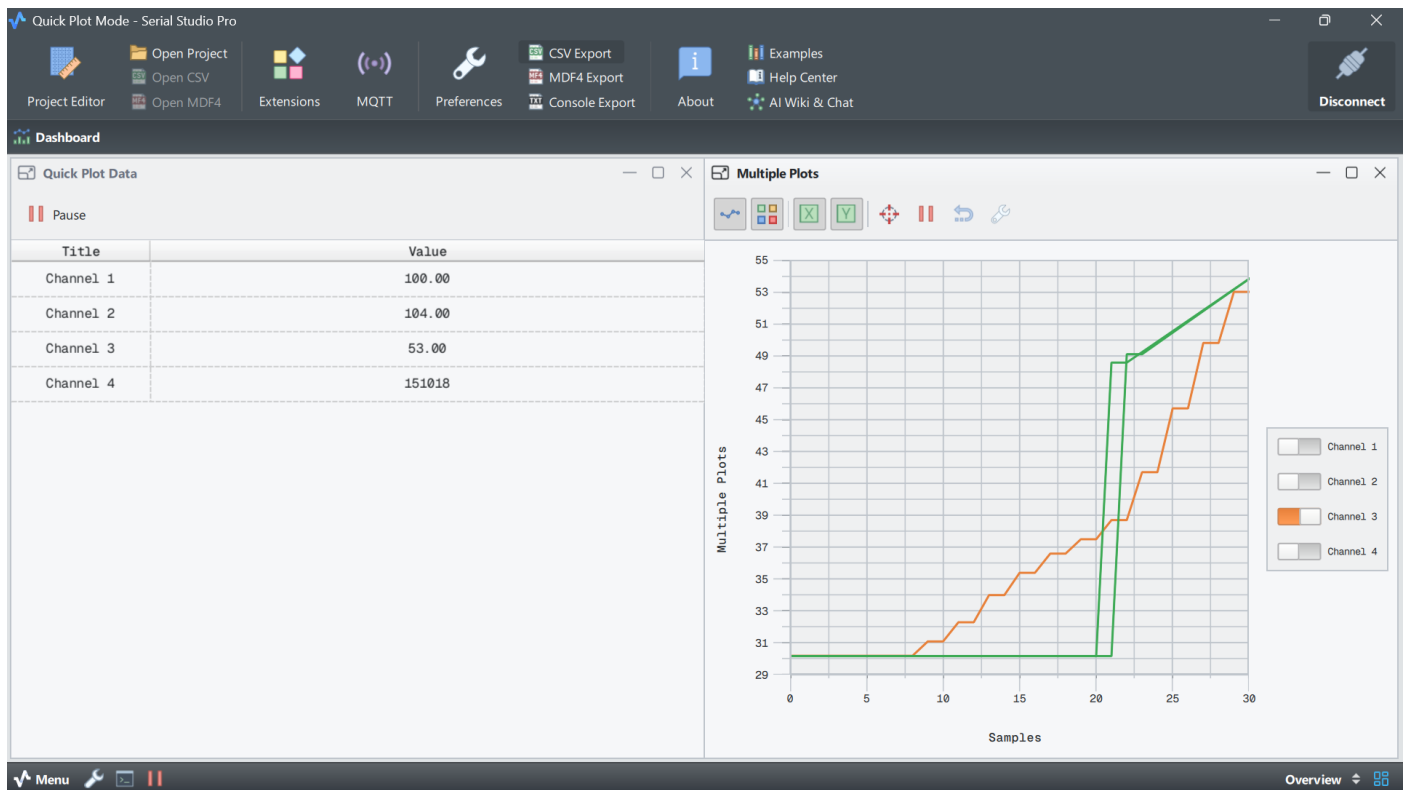


Figure 7: Timing Plot of DHT 11 Sensor

Timing Analysis of FreeRTOS Tasks:

Task Name	Priority	Trigger Type	Execution Period	Observed Response Time	Timing Observation
Radiation Task	4	Event/Threshold Based	100 ms	<100 ms	Immediate execution during radiation threshold crossing
MQ2 Task	3	Periodic	200 ms	<200 ms	Executes after Radiation Task according to priority
Temperature Task	2	Periodic	1000 ms	<1000 ms	Lower execution frequency due to lower priority
Alert Task	1	Event Based	On demand	Immediate	Activated during hazard conditions
Dashboard Task	1	Periodic	1000 ms	~1 s	Updates Streamlit and Serial Studio visualization
RTOS Tick	System	Continuous	1 ms Tick	Continuous	Confirms concurrent task scheduling

Timing Analysis Discussion:

The timing plots obtained from Serial Studio demonstrated deterministic scheduling and concurrent task execution behaviour. Radiation Task exhibited the fastest response due to the highest assigned priority. MQ2 and Temperature tasks executed according to their configured periods, while alert processing occurred immediately during threshold violations. Continuous RTOS Tick variation confirmed stable multitasking and correct scheduler operation.

CONCLUSION

The proposed RTOS-Based Auxiliary Safety Monitoring System for Nuclear Power Plant Applications was successfully designed and implemented using ESP32 and FreeRTOS. The developed system demonstrated simultaneous monitoring of radiation level, hazardous gas concentration, and environmental temperature through a multitasking architecture.

The implementation employed a potentiometer for radiation simulation, MQ-2 sensor for hazard detection, and DHT11 sensor for temperature monitoring. Multiple FreeRTOS tasks were created with priority-based scheduling to ensure deterministic behavior and timely response during hazardous situations.

The task priority mechanism:

Radiation > MQ2 > Temperature

enabled immediate processing of critical events while lower-priority tasks executed concurrently.

Queue-based communication improved synchronization between sensing and processing modules. Streamlit provided a centralized dashboard for real-time monitoring, while Serial Studio enabled timing visualization and RTOS task activity observation.

Experimental results verified:

- Simultaneous monitoring capability
- Deterministic scheduling behavior
- Automatic alert generation
- Real-time dashboard integration
- Stable multitasking operation

The developed system demonstrated suitability for low-cost safety-critical monitoring applications.

REFERENCES

- [1] E. Alves, N. Silva, W. Júnior, A. Moraes, N. Silva and A. Balieiro, “FreeRTOS Application in a Real-Time Air Quality Monitoring Architecture for Smart Campus,” *IEEE Latin America Transactions*, vol. 23, no. 4, pp. 266–273, 2025.
- [2] D. A. Janeera et al., “Smart Embedded Framework using Arduino and IoT for Real-Time Noise and Air Pollution Monitoring and Alert System,” *2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS)*, IEEE, 2021.
- [3] H. Wang, J. Huo, J. Hu and T. Wang, “Research on Real-Time Monitoring System of Multiscenario Health Data Based on Internet of Things,” *2023 International Conference on Digital Society and Intelligent Systems*, IEEE, 2023.
- [4] P. Zhu, “Design of RTOS-Based Wireless Remote Monitoring System,” *International Conference on Embedded Systems and Applications*, 2023.
- [5] C. K. Lin and B. Y. Wang, “Analyzing FreeRTOS Scheduling Behaviors with Formal Verification,” *IEEE Embedded Systems Conference*, 2022.
- [6] L. Calligaris et al., “OpenIPMC: Open Source Intelligent Platform Management Controller for Embedded Applications,” IEEE, 2020.
- [7] B. S. N. Reddy et al., “A Comprehensive Review on Functional Analysis of Real-Time Operating Systems,” *2023 International Conference on Innovative Mechanisms for Industry Applications*, IEEE, 2023.
- [8] B. Septian, M. Misbahuddin and F. Arkan, “FreeRTOS Based Air Quality Monitoring System Using Secure Internet of Things,” *Journal of Information Technology and Informatics*, vol. 3, no.1, pp.147–153, 2022.
- [9] M. H. Abdelsamea, M. Zorkany and N. Abdelkader, “Real-Time Operating Systems for Internet of Things: Vision and Research Directions,” IEEE Access.
- [10] S. Kumar and A. Jasuja, “Air Quality Monitoring System Based on Internet of Things,” *IEEE International Conference on Computing and Communication Applications*, 2021.
- [11] T. S. Gunawan et al., “Development of Smart Monitoring Architecture using RTOS for Embedded Applications,” *IEEE International Conference on Smart Instrumentation*, 2020.
- [12] L. F. Ugarte et al., “Real-Time Communication Framework for Internet of Things Devices,” *IEEE Conference on Sustainable Energy Systems*, 2020.
- [13] H. Haggi, W. Sun and J. Qi, “Resilient Monitoring Architecture for Critical Systems,” IEEE, 2020.
- [14] M. Benammar et al., “A Modular Internet of Things Platform for Real-Time Monitoring,” *IEEE Sensors Journal*, 2021.
- [15] J. Smith and R. Kumar, “Embedded Real-Time Monitoring and Scheduling for Safety-Critical Applications,” *IEEE Transactions on Industrial Informatics*, 2022.